Introduction to Java

Why Java?

- Runs on more than **5.5 billion devices**. Java is versatile and applicable to Web Development, Mobile Apps (Android), Desktop Applications, IOT Devices and more.
- Java's **OOP** implementation is widely regarded as robust and effective.
- **Enterprise and Large-Scale Systems**. Java is trusted for its stability, scalability, and security in mission-critical projects.
- **Platform Independency** allows Java to compile it’s code once and run in any OS with a JVM (Java Virtual Machine).

Platform Independency - A key feature of Java

In order to understand Platform Independency, we must know about **three core components** of Java and understand how Java executes its codes.

- ***JVM (Java Virtual Machine)*** is platform dependent interpreter. This is the component that **executes bytecode**. Mac, Linux, Windows each has different JVMs.

- ***JRE (Java Runtime Environment)*** contains the **environment and libraries** that JVM uses to execute Java programs. JRE includes JVM as well. JRE is required to RUN any java program.

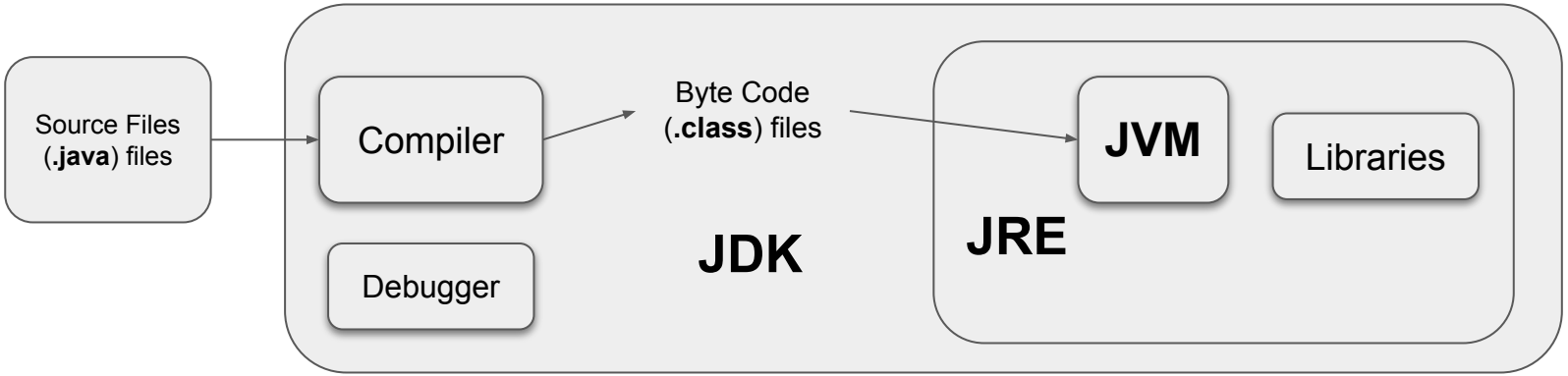
- ***JDK (Java Development Kit)*** contains various tools for developers like **Java Compiler (javac)** , **Java Debugger (jdb)** etc. JDK is required for coding and developing Java applications. JRE is included with JDK.

Platform Independency (Contd.)

1) Java code goes through the compiler within the JDK.

2) **Compiler** converts **Java Code to Byte Code** which is **platform independent**. Now this ByteCode can be run in any OS without compiling.

3) This ByteCode (.class file) is executed by the **JVM** using various libraries from the **JRE**.



Installing JDK & DrJava

For this course we'll be using JDK version 8.

Link to JDK version 8:

We'll be using DrJava as IDE:

Link to DrJava:

https://drive.google.com/file/d/1bNB0VWxQPOJ1SFmA9nFwHt_TRhm3JvVu/view

Writing Your First Program

Let's write our first program using Java

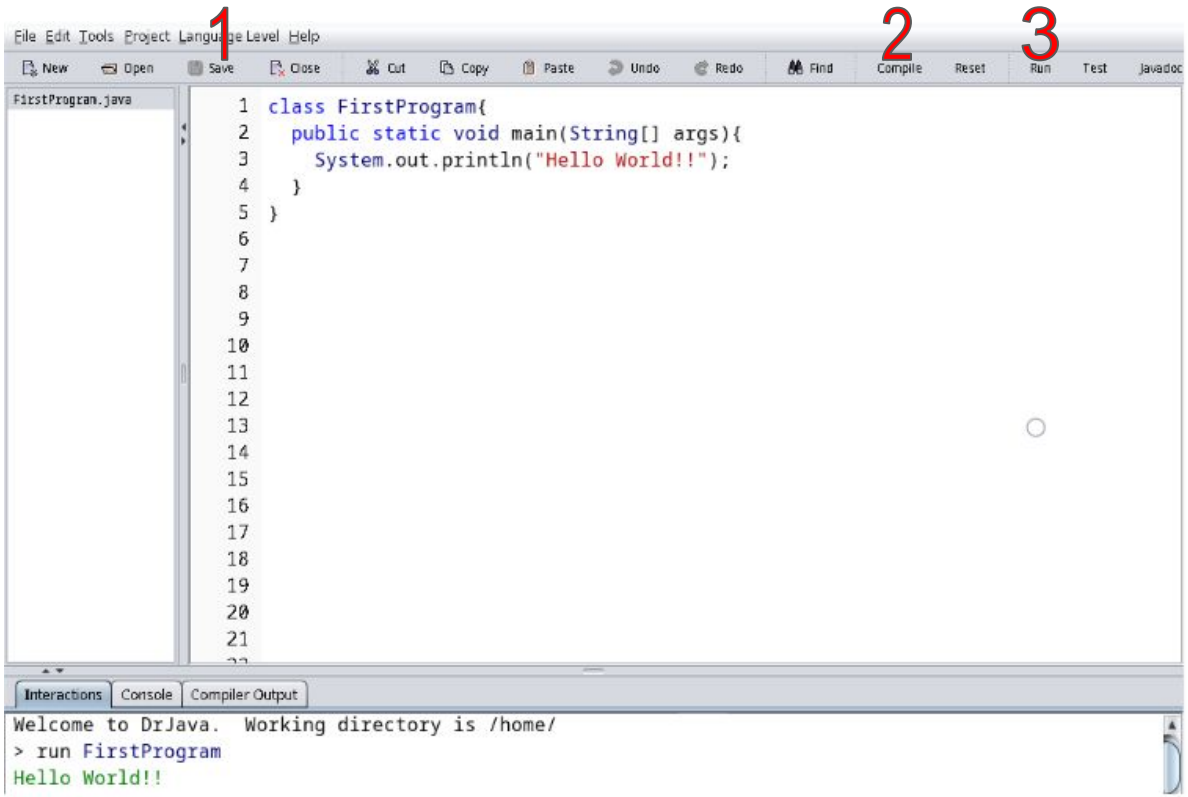
```
1.   public class FirstProgram {
2.      public static void main (String [] args){
3.         System.out.print("Hello World");
4.      }
5.   }
```

```
Welcome to DrJava.  Working directory is /home/Documents
> run FirstProgram
Hello World
```

This program will print out "Hello World".

PS: For now you have to memorize the highlighted portion.
Don't worry, all of it will make sense by the end of this course.

Running Java code using DrJava



After writing our code using Dr Java's editor, we have to

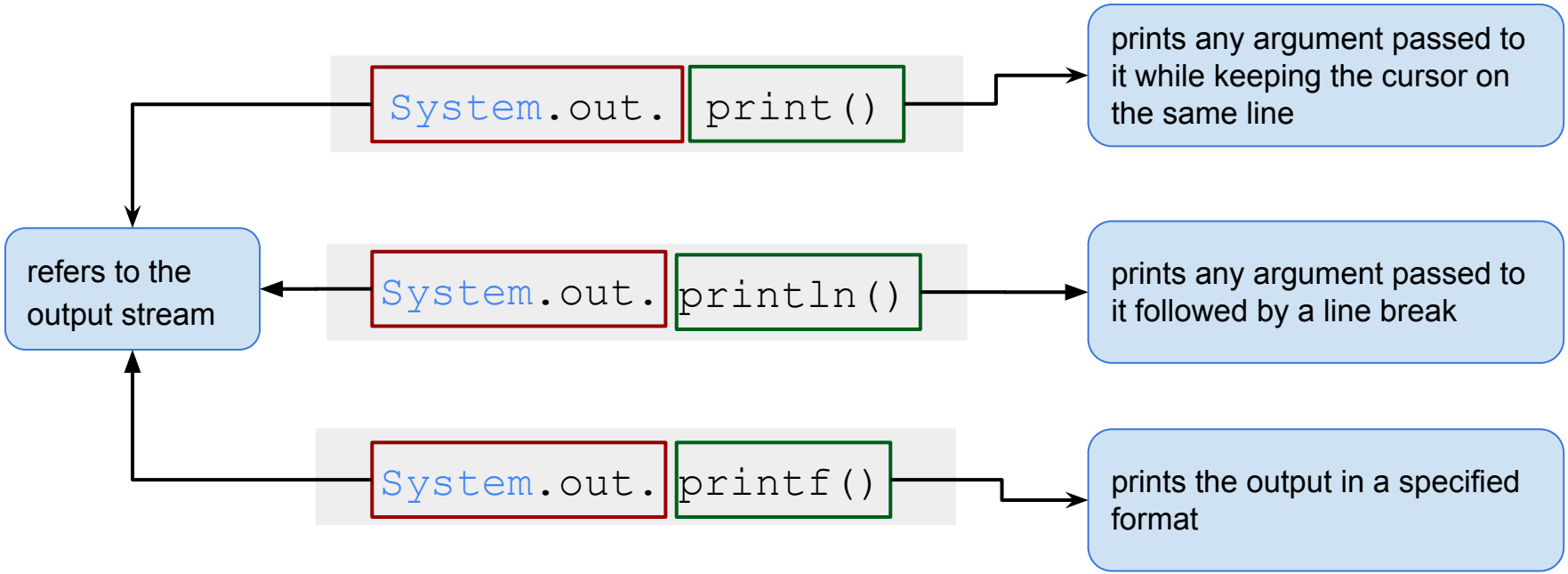
1. Save the code.
2. Compile the code.
3. Run the code.

Keep in mind, you file name has to be the same as the **Class name.**

In this example, **FirstProgram** is our class name.

Printing in Java

To show the output in the console, java uses three different approaches.



Printing in Java (Contd)

```
1.  public class PrintingExample{
2.        public static void main(String[] args)  {
3.          float y = 1.56f;
4.          System.out.print("Number: " +  23);
5.          System.out.println("Hello World");
6.          System.out.printf("Formatted y = %.4f\n", y);
7.          }
8.  }
```

```
Welcome to DrJava. Working directory is C:\Users\Desktop
> run PrintingExample
Number: 23Hello World
Formatted y = 1.5600
```

Comments in Java

- Comments are used to make the code more readable and easily understandable.
- The commented portions of code are ignored by the java compiler.
- **Single line comments:** Two forward slashes (//) are used at the beginning of the line.
- **Multi-line comments:** It starts with /* and ends with */

Comments in Java (Contd)

```
1.  public class Comments{
2.        public static void main(String[] args)   {
3.          // This is an example of single-line comment
4.          System.out.println("Comments in Java");
5.          /* This is an example of
6.           multi-line comments */
7.          System.out.println("Program ends!");
8.          }
9.  }
```

```
Welcome to DrJava. Working directory is C:\Users\Desktop
> run Comments
Comments in Java
Program ends!
```

Java Keywords

Java Keywords are few reserved words that have predefined meanings and can not be used as variable names. Let's get to know some of them.

1. **public** : An access modifier keyword. It means the class/method is accessible from any other classes. Private and Protected are other types of access modifiers.
2. **class** : keyword is used to specify the classname. In our example, class FirstProgram means that FirstProgram is the classname.

```
1.  public class FirstProgram {
2.    public static void main (String [] args){
3.      System.out.print("Hello World");
4.    }
5.  }
```

Java Keywords (Contd)

3. **void** : This keyword means nothing will be returned from the method. We'll learn more about these in the later lectures.
4. **static** : It means the method can be invoked without creating an instance of the class. Again, it'll all make sense when we learn about these in the upcoming lectures.

```
1.  public class FirstProgram {
2.    public static void main (String [] args){
3.      System.out.print("Hello World");
4.    }
5.  }
```

Java keywords list: https://www.javatpoint.com/java-keywords

Java Scopes

In Java, scopes define the visibility and lifetime of variables, methods, and classes. Primarily these scopes are defined using **{ } (Curly Braces)**.

As shown in the example, after declaring the classname “FirstProgram” in the line 1, **“{“** was used. It meant everything below was within **Class Scope** until the bracket was closed in line 5 using **“}”**.

```
1.  public class FirstProgram {
2.      public static void main (String [] args) {
3.          System.out.print(“Hello World“);
4.      }
5.  }
```

Note: We’ll learn more about scopes in the Branching and Loop chapters.

THANK YOU!